Swarm Intelligence

Smart Warehouse Autonomous Retrieval Management

Relazione di Progetto

A multi-agent system simulation for object retrieval and delivery

---

Salvatore Iurato

Corso di Swarm Intelligence
Anno Accademico 2025/26

Indice

1 Introduzione **2**
1.1 Obiettivo del problema . . . 2
1.2 Motivazione del progetto . . . 2

2 Architettura del sistema **2**
2.1 Modello dell'ambiente . . . 2
2.2 Architettura degli agenti . . . 3
2.2.1 Macchina a stati finiti . . . 3
2.3 Ruoli degli agenti . . . 4

3 Scelte implementative **4**
3.1 Mappa locale e conoscenza parziale . . . 4
3.2 Comunicazione peer-to-peer . . . 5
3.3 Pathfinding con A* . . . 5
3.4 Strategia di esplorazione degli Scout . . . 5
3.5 Gestione della batteria ed emergenza . . . 6
3.6 Visualizzazione real-time . . . 6

4 Evoluzione delle configurazioni **6**
4.1 Configurazione 1: Exploration . . . 6
4.2 Configurazione 2: Collection . . . 7
4.3 Configurazione 3: With Relay . . . 7
4.4 Tabella riassuntiva delle configurazioni . . . 8

5 Risultati sperimentali **8**
5.1 Le mappe di test . . . 8
5.2 Metriche numeriche . . . 9
5.3 Grafici comparativi . . . 10
5.4 Heatmap di visita . . . 11
5.4.1 Mappa A . . . 11
5.4.2 Mappa B . . . 12
5.5 Relay vs. struttura della mappa . . . 12

6 Conclusioni **12**
6.1 Sintesi dei risultati . . . 12
6.2 Il Relay come soluzione architetturale . . . 13
6.3 Dipendenza dalla struttura della mappa . . . 13

1. Introduzione

Il progetto implementa una simulazione di *swarm intelligence* in cui un gruppo di agenti autonomi coopera per risolvere un problema di raccolta e consegna su una mappa.

1.1. Obiettivo del problema

L'ambiente è una griglia 25×25 che rappresenta un magazzino. La griglia contiene:

- **Celle libere** percorribili dagli agenti;
- **Ostacoli** celle occupate da muri, non percorribili;
- **4 magazzini** posizionati ai quattro lati della griglia, ciascuno con un'ingresso (*entrance*) e un'uscita (*exit*) direzionali;
- **10 oggetti** sparsi nella mappa, da raccogliere e consegnare ai magazzini.

Il compito del sistema multi-agente è recuperare tutti e 10 gli oggetti nel minor numero possibile di *tick* (passi di simulazione), minimizzando al contempo il consumo energetico complessivo. Ogni agente dispone di una batteria con capacità fissa, che si riduce di un'unità ad ogni movimento. Se un agente esaurisce la batteria lontano da un magazzino, viene dichiarato *dead* e cessa di operare.

1.2. Motivazione del progetto

La sfida principale del problema è la **conoscenza parziale dell'ambiente**: ogni agente vede solo le celle entro un certo raggio visivo e costruisce una mappa locale della posizione degli oggetti noti, inizialmente vuota. La cooperazione emerge dalla condivisione delle informazioni quando due agenti sono abbastanza vicini da comunicare.

Il progetto evolve attraverso tre configurazioni successive:

- **Exploration**: priorità all'esplorazione, con più agenti Scout;
- **Collection**: priorità alla raccolta, con più agenti Collector;
- **With Relay**: architettura a tre ruoli con un Relay che funge da ponte informativo tra Scout e Collector.

Sono state testate due istanze di mappa con densità di ostacoli crescente (Mappa A e Mappa B), per analizzare come la struttura topologica dell'ambiente influenzi le performance delle diverse configurazioni.

2. Architettura del sistema

2.1. Modello dell'ambiente

L'ambiente è implementato dalla classe `Environment` e rappresenta la griglia tramite una matrice 25×25 di interi. Le celle sono classificate con i seguenti tipi:

Valore	Tipo	Descrizione
0	`EMPTY`	Cella libera percorribile
1	`WALL`	Ostacolo invalicabile
2	`WAREHOUSE`	Interno del magazzino
3	`ENTRANCE`	Ingresso unidirezionale al magazzino
4	`EXIT`	Uscita unidirezionale dal magazzino

I magazzini hanno porte direzionali, il cui verso è determinata dal lato della mappa su cui si trovano. Questa scelta modella vincoli logistici realistici, costringendo i Collector a pianificare percorsi distinti per accesso e uscita.

Gli oggetti hanno un ciclo di vita gestito interamente dall'environment:

- **Unclaimed**: visibile e prelevabile;
- **Claimed**: trasportato da un Collector;
- **Delivered**: consegnato a un magazzino, rimosso dalla mappa.

2.2. Architettura degli agenti

Tutti gli agenti ereditano dalla classe base `Agent`, che implementa:

- Una **mappa locale** 25×25; un insieme `_unobserved` traccia le celle non ancora fisicamente visitate, usato per guidare la scansione alla ricerca di oggetti;
- Un **registro degli oggetti noti** (`known_objects`), dizionario che mappa posizione e stato;
- Una **batteria** con capacità 500, decrementata di 1 ad ogni passo; al raggiungimento della soglia di emergenza l'agente torna al magazzino più vicino;
- Una **macchina a stati finiti** per il processo decisionale.

2.2.1. Macchina a stati finiti

Ogni agente ad ogni tick esegue il ciclo:

`observe()` → `communicate()` → `decide()` → `act()`

Stato	Descrizione
`EXPLORE`	Navigazione verso frontiere inesplorate
`FETCH`	Raggiungimento di un oggetto noto per prelevarlo
`DELIVER`	Trasporto dell'oggetto al magazzino più vicino
`EMERGENCY`	Rientro urgente al magazzino per batteria critica
`DEAD`	Agente inattivo (batteria esaurita lontano da magazzino)

La transizione prioritaria è: `EMERGENCY` > `DELIVER` > `FETCH` > `EXPLORE`. Lo stato di emergenza viene triggerato quando la batteria residua è inferiore alla distanza stimata fino al magazzino più vicino più un margine di sicurezza.

2.3. Ruoli degli agenti

Il sistema supporta tre ruoli distinti con parametri differenti.

Ruolo	Raggio visivo	Raggio comm.	Può raccogliere	Strategia
Scout	3	2	No	Repulsione dai pari
Collector	2	1	Sì	Garrison + burst
Relay	2	2	No	Baricentro

Scout: Ha il raggio visivo maggiore (3 celle) e si distribuisce sulla mappa usando una strategia di *repulsione*: seleziona la frontiera inesplorata che massimizza la distanza dagli altri Scout, garantendo una copertura uniforme della mappa. All'avvio, gli Scout vengono assegnati a quadranti distinti per massimizzare l'esplorazione distribuita fin dal primo tick.

Collector: Ha un raggio visivo più ridotto (2 celle) ma è l'unico agente in grado di raccogliere oggetti. Presidia inizialmente una zona di stazionamento (*garrison*) vicino a un ingresso del magazzino, da cui esegue brevi sessioni di esplorazione locale per scoprire oggetti. Il margine di emergenza è aumentato a 20 passi per garantire il rientro anche quando porta un oggetto.

Relay: Introdotto nella terza configurazione, il Relay non raccoglie oggetti ma funge da nodo di comunicazione intermedio. In condizioni normali si posiziona nel **baricentro** tra la posizione media degli Scout attivi e quella dei Collector: il target viene ricalcolato solo quando tale punto si sposta oltre una soglia, per evitare oscillazioni tick-per-tick. Così il Relay tende a collocarsi lungo la "catena" informativa tra esploratori in periferia e raccoglitori verso i magazzini. Quando ha `known_objects` non vuoti, ha priorità spostarsi verso l'ingresso del magazzino noto più vicino, per favorire l'incontro con i Collector. Solo in assenza delle posizioni degli altri agenti ricorre a un pattugliamento a diamante nel centro della mappa.

3. Scelte implementative

3.1. Mappa locale e conoscenza parziale

Ogni agente mantiene una propria rappresentazione interna dell'ambiente: una matrice 25×25, inizializzata con la griglia reale e caricata dal file JSON di input. Tutti gli agenti conoscono quindi fin dall'inizio la posizione di muri, corridoi, magazzini e porte direzionali.

La vera informazione nascosta sono le **posizioni degli oggetti**: questi non fanno parte della griglia ma sono gestiti separatamente dall'environment. Un oggetto viene reso visibile a un agente solo quando `observe()` lo rilevisce all'interno del raggio visivo, dopodiché viene aggiunto al registro locale `known_objects`.

Per tracciare cosa rimane da esplorare, ogni agente mantiene un insieme `_unobserved`: l'insieme di tutte le celle non-muro che non sono ancora state attraversate entro il raggio visivo. Gli Scout si dirigono verso queste celle non per scoprire la topologia,

ma per passarci abbastanza vicino da rivelare eventuali oggetti nascosti. La fase di esplorazione è dunque in realtà una fase di *scansione* del territorio alla ricerca di oggetti.

3.2. Comunicazione peer-to-peer

Quando due agenti si trovano entro il raggio di comunicazione (*comm_radius*), eseguono un *merge* bidirezionale delle rispettive conoscenze. Poiché la struttura della mappa è già nota a tutti, il merge riguarda essenzialmente due informazioni:

- Il **registro degli oggetti noti** (`known_objects`): gli oggetti osservati da uno Scout vengono trasmessi all'altro agente, rendendoli immediatamente disponibili per il fetch;
- L'**insieme delle celle non ancora visitate** (`_unobserved`): viene sostituito con l'intersezione dei due insiemi, così le celle già scansionate da uno non vengono ri-visitate dall'altro, evitando ridondanza.

Vengono anche condivisi i `claimed_frontier`, le celle-obiettivo già prese in carico, così gli Scout si coordinano per non dirigersi verso la stessa zona.

Questa comunicazione locale, senza infrastruttura centralizzata, è il meccanismo attraverso cui l'intelligenza collettiva emerge: uno Scout che ha scansionato il quadrante nord-est trasferisce immediatamente quella conoscenza a qualsiasi agente che gli si avvicini.

3.3. Pathfinding con A*

Il modulo `pathfinding.py` implementa l'algoritmo A* con euristica di distanza di Manhattan. L'algoritmo riceve una funzione `is_walkable_fn` che opera sulla mappa locale dell'agente anziché sulla mappa reale: questo garantisce che la pianificazione sia coerente con la conoscenza parziale disponibile.

La scelta di A* è motivata dalla necessità di gestire rotte ottimali in presenza di ostacoli complessi e passaggi stretti, rispettare i vincoli direzionali di ingresso/uscita dei magazzini e operare efficientemente su una griglia di dimensione 25×25. Un algoritmo come BFS sarebbe comunque accettabile in uno scenario simile, ma A* offre un margine migliore in presenza di molti muri.

Quando A* non trova un percorso (es. la mappa locale non ha ancora rivelato il collegamento verso la destinazione), l'agente si sposta casualmente verso la destinazione in attesa di scoprire nuove celle.

3.4. Strategia di esplorazione degli Scout

Gli Scout adottano una strategia di *coverage-based scanning* con repulsione inter-agente. L'obiettivo è selezionare la cella non ancora visitata (`_unobserved`) che:

1. È raggiungibile tramite A*;
2. Massimizza la distanza media dagli altri Scout attivi.

Questo garantisce una **scansione uniforme e non ridondante** dell'area. Senza repulsione, più Scout tenderebbero a convergere sulle stesse zone, lasciando ampie aree non scansionate e quindi con oggetti non ancora scoperti.

All'avvio, gli Scout vengono assegnati a quadranti distinti della griglia (angolo superiore-sinistro, superiore-destro, ecc.) mediante un target iniziale fisso, accelerando la diversificazione geografica nei primi tick.

3.5. Gestione della batteria ed emergenza

Ogni agente tiene traccia della batteria residua e calcola ad ogni tick la distanza stimata fino al magazzino più vicino accessibile. Se:

$$\text{batteria} \leq \text{distanza_magazzino} + \text{margine_sicurezza}$$

l'agente entra in stato `EMERGENCY` e abbandona il task corrente per tornare al magazzino, indipendentemente da ciò che stava facendo (incluso trasportare un oggetto).

Il margine di sicurezza è differenziato per ruolo: i Collector hanno un margine maggiore (20 passi) rispetto agli Scout (10 passi), poiché il rischio di perdere un oggetto in transito è più costoso del rischio di perdere uno Scout esploratore.

Gli agenti che muoiono lontano da un magazzino (batteria zero senza essere rientrati) vengono marcati come `DEAD` e non emettono più azioni.

3.6. Visualizzazione real-time

L'interfaccia grafica è implementata con PySide6: la finestra principale aggiorna la mappa dal punto di vista dell'**unione delle mappe locali** di tutti gli agenti, cioè ciò che il collettivo conosce nell'istante corrente. Gli oggetti non ancora scoperti non appaiono, permettendo di osservare l'esplorazione collettiva emergere tick per tick.

Ogni agente è rappresentato con un colore specifico (verde per Scout, arancione per Collector, viola per Relay, grigio per Dead), e le celle di ingresso/uscita dei magazzini mostrano frecce direzionali. Il pannello laterale mostra le metriche in tempo reale: tick corrente, batteria di ciascun agente, oggetti consegnati e rimanenti.

4. Evoluzione delle configurazioni

Il design del sistema ha seguito un percorso iterativo in cui ogni configurazione ha affrontato i limiti di quella precedente. Il team di agenti è sempre composto da 5 unità, ma la distribuzione dei ruoli cambia radicalmente le performance osservate.

4.1. Configurazione 1: Exploration

La prima iterazione puntava sulla **massima copertura della mappa**, con 3 Scout e soli 2 Collector. L'ipotesi di partenza era che la fase limitante del processo fosse la scoperta degli oggetti: una volta noti tutti i target, i Collector avrebbero potuto raccoglierli rapidamente.

L'ipotesi si è rivelata parzialmente errata. Nelle esecuzioni sperimentali, la mappa viene esplorata in tempi ragionevoli dai 3 Scout, ma i 2 soli Collector non riescono a sfruttare appieno l'informazione prodotta: devono coprire da soli tutti e 10 gli oggetti con percorsi lunghi, senza supporto aggiuntivo per la raccolta. Inoltre, il canale di comunicazione Scout - Collector è indiretto e dipende dall'incontro fisico casuale tra agenti, il che introduce ritardi significativi: uno Scout può aver scoperto un oggetto ma il Collector relativo non ne viene a conoscenza fino a molto tempo dopo.

Nei run salvati, su Mappa B i tick totali restano più alti per *Exploration* (131) rispetto a *Collection* e *With Relay* (105 e 103), con energia per oggetto maggiore (64,7 vs. circa 52). Su Mappa A invece questi stessi run mostrano *Exploration* al 127 tick, valore inferiore alle altre due configurazioni (141 e 139): il trade-off Scout/Collector non si traduce sempre in un peggioramento temporale su mappe aperte, mentre su topologia labirintica la mancanza di Collector dedicati penalizza di più.

4.2. Configurazione 2: Collection

La seconda iterazione inverte la proporzione: 2 Scout e 3 Collector. L'obiettivo è avere più unità disponibili per la raccolta, sacrificando un'unità di esplorazione.

Su Mappa B i tempi migliorano nettamente rispetto a *Exploration*: 105 tick e 52,5 energia/oggetto. Su Mappa A i run correnti riportano 141 tick e 67,5 energia/oggetto, peggiori dell'Exploration su quella mappa: i 3 Collector accelerano il parallelismo della raccolta quando l'informazione è disponibile, ma con 2 Scout la scoperta e gli spostamenti possono ritardare il completamento globale rispetto alla tripla esplorazione. Con 3 Collector che presidiano zone diverse della griglia, gli oggetti vengono comunque raccolti in parallelo non appena le posizioni sono note.

Il problema della propagazione dell'informazione rimane. Con 2 soli Scout, la scoperta di oggetti in zone lontane dai Collector può tardare. Soprattutto su Mappa B (più ostacoli), gli Scout devono percorrere corridoi complessi e l'incontro fisico con i Collector non è garantito in tempi brevi. La comunicazione diretta Scout - Collector dipende dalla topologia della mappa e dalla traiettoria degli agenti.

4.3. Configurazione 3: With Relay

La terza configurazione introduce il ruolo **Relay**: un agente specializzato esclusivamente nella *propagazione dell'informazione*. Non raccoglie oggetti e non esplora sistematicamente le frontiere: di norma si colloca sul **baricentro** tra Scout e Collector attivi, fungendo da nodo intermedio nella catena di comunicazione; quando possiede già informazioni su oggetti si avvicina agli ingressi dei magazzini noti. Solo in assenza delle posizioni degli altri agenti ricorre, come fallback, a un pattugliamento a diamante nel centro della mappa.

Motivazione del design. Di norma, uno Scout scopre un oggetto in periferia, ma per trasmettere questa informazione a un Collector deve avvicinarsi fisicamente entro 1 cella di *comm_radius* (il Collector ha raggio 1). Questo non sempre avviene in tempi brevi.

Il Relay risolve il problema aumentando il "raggiungimento" della rete: avendo *comm_radius* 2 (uguale agli Scout), può raccogliere informazioni dagli Scout nelle zone centrali-periferiche e trasmetterle ai Collector vicino ai magazzini in un secondo passaggio. In pratica il Relay trasforma la topologia di comunicazione da un grafo casuale dipendente dalle traiettorie a una struttura dove il centro funge da punto di incontro garantito.

Si perde un Collector, il che riduce il parallelismo nella raccolta. Il Relay è quindi giustificato solo se il *guadagno nella velocità di propagazione* delle informazioni compensa la perdita di capacità di raccolta. Nei run attuali *With Relay* impiega 2 tick in meno di *Collection* su entrambe le mappe (139 vs. 141 su A; 103 vs. 105 su B), suggerendo che il routing via hub centrale compensa in questi scenari il Collector in meno.

4.4. Tabella riassuntiva delle configurazioni

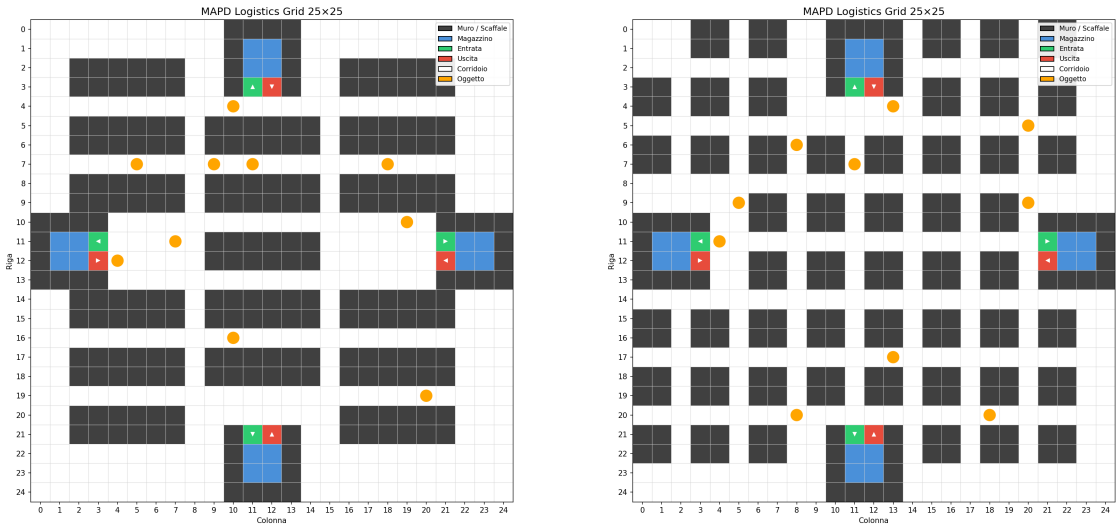
Configurazione	Scout	Collector	Relay	Focus
Exploration	3	2	0	Copertura mappa
Collection	2	3	0	Raccolta parallela
With Relay	2	2	1	Info routing

5. Risultati sperimentali

Tutti e 6 gli scenari (2 mappe × 3 configurazioni) sono stati eseguiti con un limite massimo di 500 tick. Il confronto si concentra su efficienza temporale ed energetica.

5.1. Le mappe di test

Le due istanze di mappa rappresentano ambienti con complessità crescente.



(a) Mappa A (b) Mappa B

Figura 1: Visualizzazione delle due istanze di mappa. I quadrati colorati ai bordi sono i magazzini; i punti dorati sono gli oggetti da raccogliere.

La mappa A presenta una struttura relativamente aperta, con ostacoli distribuiti in modo da non creare colli di bottiglia significativi. Gli agenti possono muoversi quasi liberamente tra i quadranti. La Mappa B ha invece una densità di muri notevolmente superiore, con corridoi stretti che costringono gli agenti a percorsi obbligati e rendono più difficile sia l'esplorazione sia l'incontro tra agenti per la comunicazione.

5.2. Metriche numeriche

Tabella 1: Risultati sperimentali – Mappa A

Configurazione	**Tick totali**	**Energia media**	**Energia/oggetto**	**Tick/oggetto**
Exploration	127	127	63.5	12.7
Collection	141	135	67.5	14.1
With Relay	139	134.6	67.3	13.9

Tabella 2: Risultati sperimentali – Mappa B

Configurazione	**Tick totali**	**Energia media**	**Energia/oggetto**	**Tick/oggetto**
Exploration	131	129.4	64.7	13.1
Collection	105	105	52.5	10.5
With Relay	103	103	51.5	10.3

Lettura dei dati. I valori in tabella provengono dagli ultimi run salvati in `experiments/.../results.json`. Su Mappa B la configurazione *Exploration* è la meno efficiente in tempo ed energia per oggetto (131 tick; 64,7 energia/oggetto), mentre *Collection* e *With Relay* convergono a tempi molto più bassi (105 e 103 tick), con *With Relay* leggermente migliore su entrambe le metriche aggregate.

Su Mappa A il quadro è diverso: in questi run *Exploration* completa in 127 tick con 63,5 energia/oggetto, risultato migliore di *Collection* (141 tick, 67,5) e di *With Relay* (139 tick, 67,3). Ciò mostra che l'efficacia non è monotona rispetto al numero di Collector: su mappa aperta i 3 Scout e i percorsi brevi possono far terminare la missione prima, pur con un profilo organizzativo diverso; su mappa labirintica il collo di bottiglia diventa la raccolta e la comunicazione, dove più Collector e il Relay aiutano di più.

Collection e *With Relay* restano vicine: su Mappa A il Relay richiede 2 tick in meno della Collection (139 vs. 141, circa −1,4%); su Mappa B ancora 2 tick in meno (103 vs. 105, circa −1,9%).

5.3. Grafici comparativi

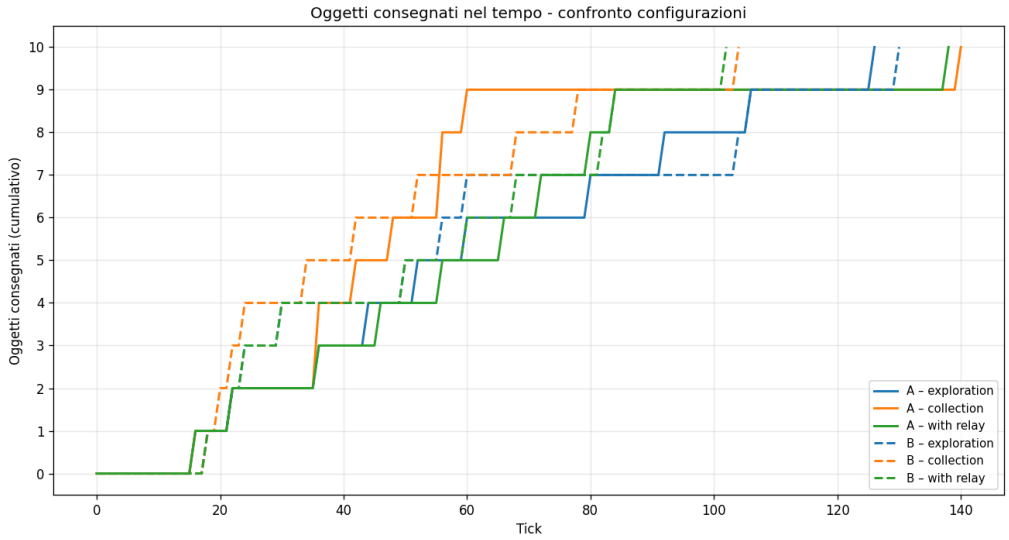


Figura 2: Numero cumulativo di oggetti consegnati nel tempo, per tutte le configurazioni e mappe.

Il grafico confronta le curve cumulative: su Mappa B le tracce di *Exploration* raggiungono il completamento più tardi rispetto a *Collection* e *With Relay*; su Mappa A le pendenze relative dipendono dal run e vanno lette insieme alle tabelle.

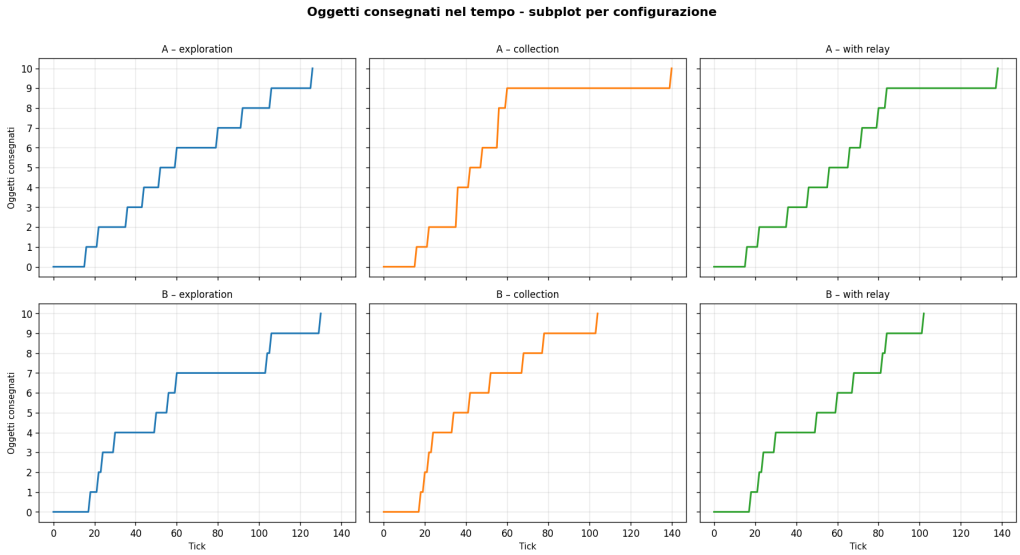


Figura 3: Griglia di subplot: Ogni pannello mostra la curva cumulativa degli oggetti consegnati per quella combinazione.

Osservando i subplot per istanza, su Mappa B *With Relay* completa in 103 tick e *Collection* in 105: le curve cumulative possono sovrapporsi o incrociarsi nel tratto intermedio, ma i tick totali in tabella differiscono leggermente.

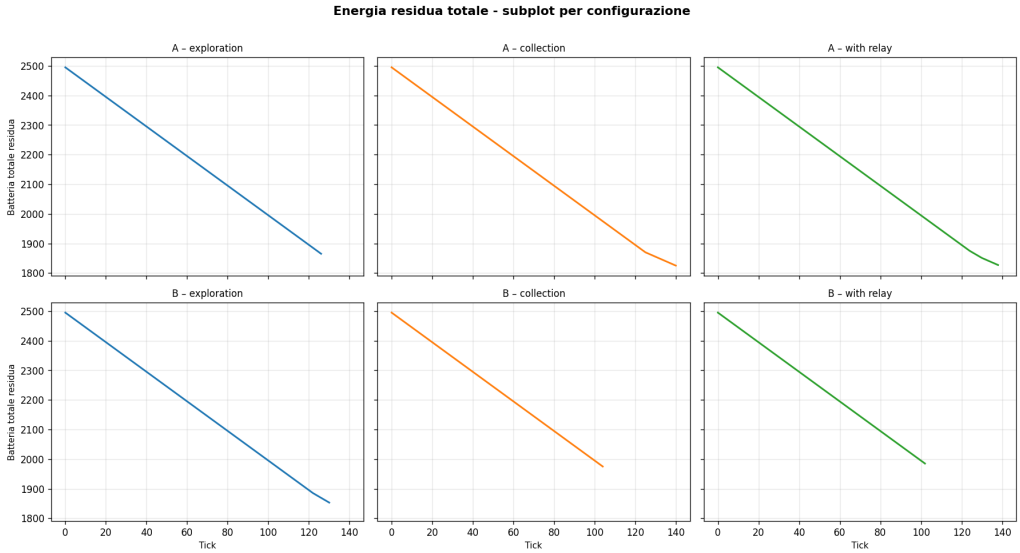


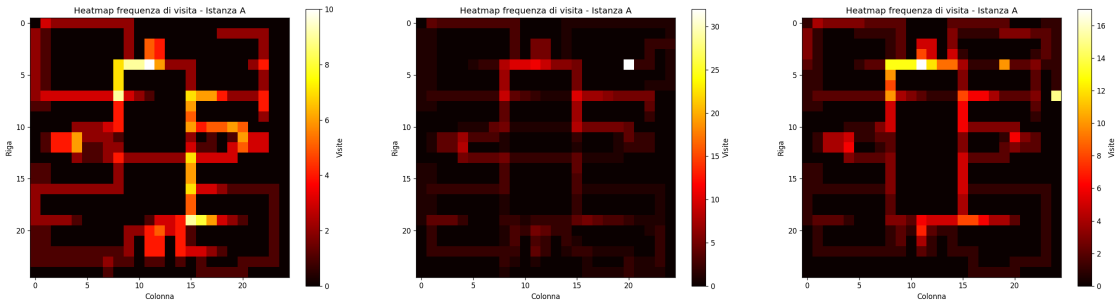
Figura 4: Griglia di subplot per il consumo energetico: ogni pannello mostra il profilo di consumo di quella combinazione istanza-configurazione.

Il profilo energetico è lineare per costruzione (ogni agente consuma 1 unità per tick finché è attivo). La somma delle energie consumate segue quindi l'andamento della durata della simulazione; su Mappa B le configurazioni con meno tick mostrano totali più bassi a fine run.

5.4. Heatmap di visita

Le heatmap mostrano la frequenza di visita di ciascuna cella della griglia aggregata su tutti gli agenti e tutti i tick della simulazione. Zone più calde indicano celle frequentate spesso, zone fredde e più scure celle raramente visitate.

5.4.1. Mappa A



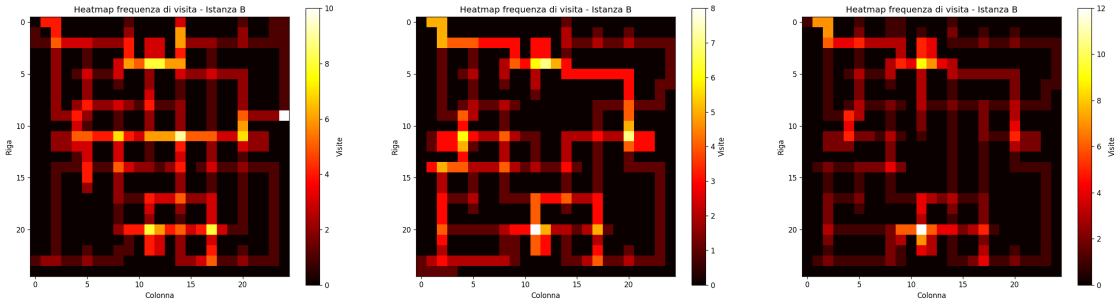
(a) Exploration (b) Collection (c) With Relay

Figura 5: Heatmap di visita per Mappa A nelle tre configurazioni.

Su Mappa A, la configurazione *Exploration* produce heatmap con una distribuzione più uniforme della presenza agenti sull'intera griglia: gli Scout diffondono sistematicamente la copertura su tutta la mappa. La configurazione *Collection* mostra invece una concentrazione più elevata nelle aree prossime ai magazzini e agli oggetti, con

meno ridondanza nelle zone già esplorate. La configurazione *With Relay* evidenzia concentrazione di attività verso il *centro* della mappa e lungo i collegamenti agli ingressi dei magazzini, in linea con il posizionamento sul baricentro Scout–Collector e con le rotte verso i magazzini quando gli oggetti sono già noti.

5.4.2. Mappa B



(a) Exploration (b) Collection (c) With Relay

Figura 6: Heatmap di visita per Mappa B nelle tre configurazioni.

Su Mappa B l'effetto della densità degli ostacoli è evidente: le heatmap mostrano corridoi obbligati di transito ad alta frequenza, separati da zone completamente fredde (i muri). La configurazione *With Relay* produce una distribuzione più bilanciata dell'attività lungo i corridoi centrali, a conferma del ruolo di ponte svolto dall'agente Relay tra le zone periferiche degli Scout e le aree magazzino dei Collector.

5.5. Relay vs. struttura della mappa

Nei run registrati, ***With Relay*** batte ***Collection*** per pochi tick su entrambe le mappe (Mappa A: 139 vs. 141; Mappa B: 103 vs. 105), pur avendo un Collector in meno. Il margine è modesto ma coerente con un relay che snellisce la propagazione delle informazioni.

Nella Mappa A, la differenza Relay–Collection è di 2 tick ($\approx 1{,}4\%$): in ambiente relativamente aperto gli incontri diretti sono frequenti, quindi il vantaggio del nodo centrale è limitato rispetto al costo operativo di sostituire un Collector con il Relay.

Su Mappa B il divario resta di 2 tick ($\approx 1{,}9\%$) a favore del Relay: corridoi stretti e ostacoli rendono più utile un hub di passaggio, in linea con la motivazione progettuale.

Su mappe ancora più frammentate è plausibile che il margine del Relay cresca; nei dati attuali il guadagno è piccolo ma costante rispetto a *Collection*.

6. Conclusioni

6.1. Sintesi dei risultati

Il progetto ha sviluppato e confrontato tre configurazioni di uno sciame eterogeneo per un problema di raccolta e consegna su griglia. Dai `results.json` degli ultimi run non emerge una gerarchia fissa: su Mappa B *Exploration* è la meno efficiente

(131 tick), mentre su Mappa A in questi dati è la più veloce (127 tick) rispetto a *Collection* (141) e *With Relay* (139).

Resta valida l'analisi qualitativa: pochi Collector e comunicazione solo per incontro fisico possono limitare la raccolta; il numero effettivo di tick dipende però in modo forte dalla topologia e dalla dinamica del run.

Collection e *With Relay* restano vicine, con meccanismi diversi (parallelismo vs. routing), e nei dati attuali il Relay termina leggermente prima della Collection su entrambe le mappe.

6.2. Il Relay come soluzione architetturale

Il contributo principale della configurazione *With Relay* non è la superiorità numerica bruta, ma l'**architettura di comunicazione più robusta**. Il Relay trasforma la topologia della rete da un grafo casuale e dipendente dalle traiettorie degli agenti a una struttura con un nodo centrale garantito.

Questa robustezza architetturale è particolarmente rilevante su Mappa B: con un Collector in meno (2 vs. 3), *With Relay* completa in 103 tick contro 105 di *Collection*. Il passaggio delle informazioni sugli oggetti risulta meno legato al caso degli incontri diretti Scout–Collector. Il Relay offre un percorso comunicativo strutturato (hub centrale) che *favorisce* il flusso informativo rispetto a una rete puramente ad hoc.

6.3. Dipendenza dalla struttura della mappa

Il risultato forse più importante del progetto è la dimostrazione che **non esiste una configurazione universalmente ottimale**: la scelta dipende dalla topologia della mappa.

- **Mappa aperta con pochi ostacoli**: nei run attuali *Exploration* ha i tick più bassi; *With Relay* resta leggermente migliore di *Collection* in tempo (139 vs. 141). La scelta del ruolo dipende quindi da obiettivi oltre al solo conteggio tick (robustezza, predicibilità).
- **Mappa complessa con ostacoli**: *Exploration* è nettamente più lenta; *With Relay* (103 tick) migliora *Collection* (105) pur con un Collector in meno, riducendo la dipendenza dagli incontri casuali lungo i corridoi.
- **Mappa estremamente complessa** (ipotetica): è plausibile che il margine del Relay rispetto a sole strategie dirette cresca ulteriormente, fino a giustificare più relay o una rete distribuita, ipotesi non verificata dai run attuali.

Questa dipendenza dalla struttura è un risultato fondamentale nel campo della swarm intelligence: le strategie di coordinamento non sono universali ma devono essere progettate in relazione all'ambiente di riferimento. Il Relay è, in questo senso, una risposta adattiva alla sfida comunicativa imposta dalla topologia.